

Analiza stabilności algorytmów sumowania w zależności od precyzji typu danych

Artur Radwański

12 marca 2026

1 Środowisko

- System operacyjny: Linux 6.17.9-arch1-1
- Procesor: Intel Core i5-7200U
- Język programowania: c++
- Wersja kompilatora: gcc version 15.2.1 20260209 (GCC)

2 Algorytm sumacyjny Kahana

Algorytm Kahana pozwala na znaczną redukcję błędu zaokrągleń podczas sumowania ciągu liczb zmiennoprzecinkowych. Dla każdego elementu wejściowego x_i , przy założeniu wartości początkowych $s_0 = 0$ oraz $c_0 = 0$ (poprawka), wykonuje się następujące kroki:

1. **Obliczenie poprawionej wartości:**

$$y_i = x_i - c_{i-1}$$

Od bieżącego elementu odejmujemy zakumulowany błąd z poprzedniej iteracji.

2. **Wyznaczenie sumy tymczasowej:**

$$t_i = s_{i-1} + y_i$$

W tym kroku następuje właściwa operacja dodawania, w której mniejsze bity y_i mogą zostać odcięte.

3. **Obliczenie nowej poprawki:**

$$c_i = (t_i - s_{i-1}) - y_i$$

Obliczamy różnicę między tym, co faktycznie zostało dodane do sumy ($t_i - s_{i-1}$), a wartością, którą zamierzaliśmy dodać (y_i).

4. **Aktualizacja sumy głównej:**

$$s_i = t_i$$

Dzięki takiemu podejściu, błąd odcięty w kroku 2. zostaje "odrodzony" w zmiennej c_i i uwzględniony (z przeciwnym znakiem) w kolejnej iteracji, co pozwala zachować precyzję zbliżoną do typów o wyższej rozdzielczości (np. `double` dla danych typu `float`).

3 Wyniki eksperymentu

W tabeli 1 zestawiono wyniki sumowania 1.0 oraz 10^6 składników o wartości 10^{-6} dla trzech różnych typów precyzji.

Algorytm	float (32-bit)	double (64-bit)	long double (80-bit)
Wartość oczekiwana	2.0000000000	2.0000000000	2.0000000000
Dodawanie naturalne	1.9536743164	1.9999999999	2.0000000000
Suma Kahana	2.0000000000	2.0000000000	2.0000000000
Sumowanie (HeapSort)	2.0090389252	2.0000000000	2.0000000000

Tabela 1: Porównanie wyników sumowania w zależności od typu danych.

4 Analiza błędów i uwarunkowanie zadania

4.1 Uwarunkowanie zadania

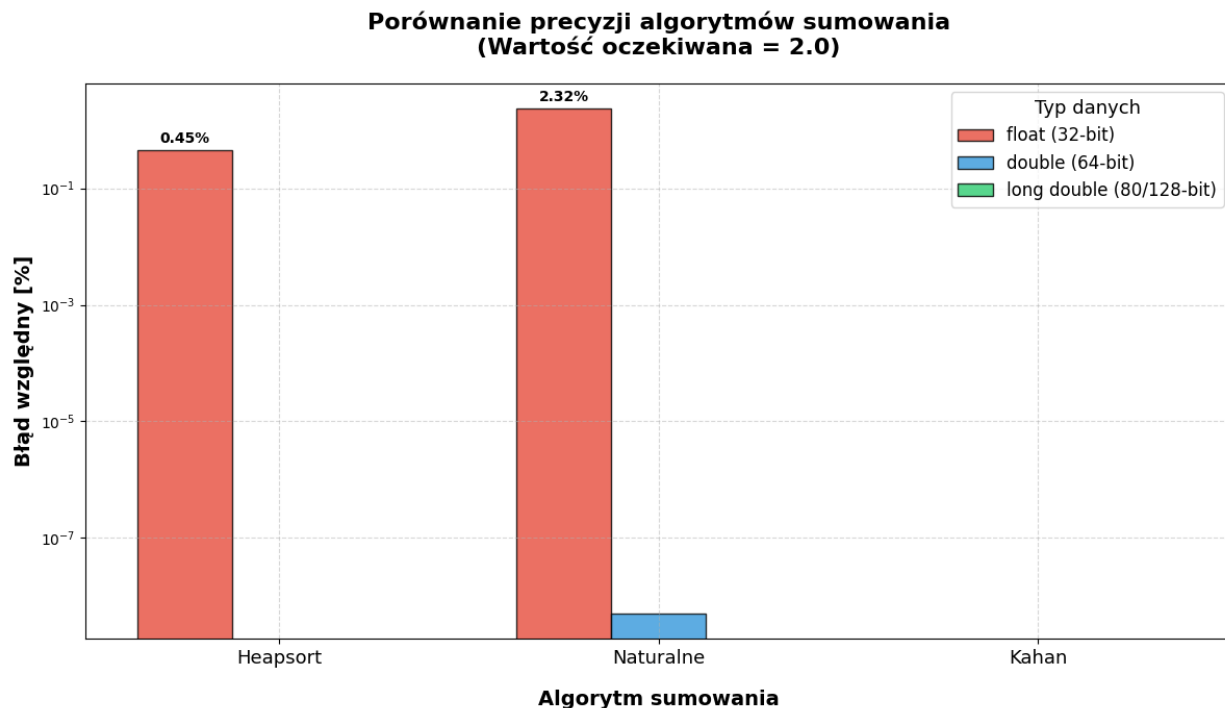
Zadanie sumowania liczb o drastycznie różnych rzędach wielkości (1.0 oraz 10^{-6}) jest klasycznym przykładem **złego uwarunkowania numerycznego**. W arytmetyce zmiennoprzecinkowej IEEE 754, aby dodać dwie liczby, ich wykładniki muszą zostać wyrównane, co powoduje przesunięcie mantysy mniejszej liczby w prawo.

4.2 Analiza wyników dla typu float

- **Dodawanie naturalne (1.9536):** Wynik jest mniejszy od oczekiwanego z powodu zjawiska **absorpcji**. Gdy suma częściowa rośnie, mantysa liczby 10^{-6} traci najmłodsze bity. W pewnym momencie składnik staje się "za mały", by zmienić wartość sumy, co generuje stały błąd systematyczny.
- **Algorytm Kahana (2.0000):** Wykazuje najwyższą stabilność. Wykorzystuje zmienną poprawki c do przechowywania utraconych bitów (ang. *lost low-order bits*) i uwzględnia je w następnej iteracji, co pozwala na osiągnięcie precyzji maszynowej bez sortowania.
- **HeapSort (2.0090):** Wynik powyżej 2.0 sugeruje błąd typu overflow przy arytmetyce float.

4.3 Wpływ precyzji (double i long double)

Przejdzie na **double** zwiększa mantysę z 24 do 53 bitów. Błąd absorpcji dla 10^6 operacji przesuwają się poza zakres widoczny w standardowym wypisywaniu (pojawia się dopiero jako 1.9999999999). Dla **long double** precyzja jest na tyle wysoka, że błąd zaokrągleń dla tej skali danych jest nieobserwowalny.



Rysunek 1: Wykres błędu względnego otrzymanego wyniku w porównaniu do wyniku teoretycznego

5 Analiza czasu wykonywania

Analiza czasu wykonywania wskazuje na istotne różnice w koszcie operacji zmiennoprzecinkowych oraz dostępu do pamięci, zwłaszcza w przypadku long double.

Algorytm	float [s]	double [s]	long double [s]
Dodawanie naturalne	0.003466	0.003520	0.003962
Suma Kahana	0.011358	0.011532	0.016004
Sumowanie (HeapSort)	0.026127	0.027380	0.045618

Tabela 2: Czas wykonania algorytmów dla 10⁶ elementów [sekundy].

5.1 Interpretacja wyników wydajnościowych

- Efektywność sumowania naturalnego:** Jest to najszybsza metoda, ponieważ procesor wykonuje tylko jedną operację dodawania na element ($sum + = x$). Bardzo niski czas dla double sugeruje optymalne wykorzystanie instrukcji wektorowych (SIMD) przez kompilator.
- Koszt Algorytmu Kahana:** Kahan jest około 2-3 razy wolniejszy od sumowania naturalnego. Wynika to z faktu, że na każdy element tablicy przypadają cztery operacje zmiennoprzecinkowe, które tworzą łańcuch zależności (ang. *dependency chain*), uniemożliwiając pełne równoległe przetwarzanie wewnątrz rdzenia.
- Złożoność HeapSort:** Jest to metoda zdecydowanie najwolniejsza ($O(n \log n)$). Co ciekawe, czas dla double (0.16s) jest znacznie wyższy niż dla float, co wynika z większego obciążenia szyny pamięci oraz faktu, że dane typu double zajmują dwa razy więcej miejsca w Cache L2/L3.